

Как отправить домашку через Planerverse

Здесь вы регистрируете участие в coding agents contest и отправляете репозиторий агента на проверку. Отдельный аккаунт на Planerverse создавать не нужно: на [странице участия](#) есть открытая форма заявки.

Дедлайн: 29 октября 2026, 23:59 МСК. Все агенты на стенде используют **DeepSeek V4.1 Flash, thinking high**. Временный API-ключ предоставляет стенд; покупать или вставлять свой ключ в форму не нужно. Основной прогон после отбора включает 30 задач SWE-bench. [Правила](#) и [технический контракт](#).

Шаг 1 Подготовьте агента

Возьмите содержимое `homework_02_agent/` из архива либо доработайте свой агент. Открытый исходный проект использовать можно, но нужны собственные изменения.

В заготовке уже есть четыре инструмента: `list_files`, `search_code`, `read_file`, `replace_in_file`. Реализуйте инструкции и цикл «запрос модели, вызовы инструментов, результаты, новый запрос». Сохраняйте историю на стороне своей программы, сопоставляйте вызовы и результаты по `call_id`, ограничьте число шагов и общее время.

Для текущего демо обработайте также `status == "incomplete"` и `incomplete_details`. Для Chat Completions аналогичный случай обозначается `finish_reason == "length"`. Не разбирайте пустой или оборванный ответ как завершённый JSON и не выполняйте неполные аргументы вызова. Ограничьте повторные попытки; уменьшайте ненужный контекст и размер запрашиваемого фрагмента. Инструкцию платформы можно передать целиком своему coding-агенту. [Контракт запуска и ограничения](#).

Для своей интеграции следуйте её протоколу обработки неполных ответов: нельзя просто дописать произвольный `function_call_output` без соответствующего полного вызова. Не уменьшайте контекст так, чтобы потерять связь уже сохранённых вызовов и результатов.

Шаг 2 Создайте публичный GitHub репозиторий

Если аккаунта GitHub ещё нет, зарегистрируйте его на [GitHub](#). Создайте репозиторий с видимостью **Public**. Код можно загрузить через GitHub Desktop, `git` или веб-интерфейс GitHub. В веб-интерфейсе используйте загрузку файлов в репозиторий; сохраняйте структуру папок и не выбирайте настоящие `.env`. Для заготовки курса корень репозитория должен выглядеть так:

```
agent.json
agent.py
pyproject.toml
uv.lock
README.md
.gitignore
.env.example
local_check/
```

Загружайте содержимое `homework_02_agent/`, а не родительскую папку. Важно: платформа ищет `agent.json` в корне. Для другого стека используйте его lock-файл и согласованный способ запуска.

Перед публикацией проверьте файлы. Не отправляйте настоящие `.env`, API-ключи, токены, приватные данные и логи с секретами. `.env.example` должен содержать только примеры или пустые значения. Если секрет уже попал в коммит, простого удаления файла недостаточно: отзовите ключ и очистите историю перед публикацией.

Сделайте коммит и отправьте его на GitHub. Полный SHA получите командой из папки репозитория:

```
git rev-parse HEAD
```

Это строка из 40 шестнадцатеричных символов, а не короткий SHA, имя ветки или ссылка на папку. Проверьте, что именно этот коммит доступен в публичном репозитории. После правок нужен новый коммит, push и новый SHA. [Требования к репозиторию](#).

Шаг 3 Проверьте локальный запуск

В заготовке есть локальная проверка:

```
uv sync
uv run python local_check/run_check.py
```

Для неё задайте параметры модели согласно `homework_02_agent/README.md`; локальные запросы используют ваши настройки и могут быть платными. Локальная проверка не заменяет новое демо: большой контекст и искусственный незавершённый ответ проверяются на сайте отдельно.

На стенде код агента располагается в `/agent`, задача в `/task/TASK.md`, редактируемый репозиторий в `/testbed.entrypoint` в `agent.json` задаётся массивом аргументов, а не `shell`-строкой. Исправления должны появиться в рабочем репозитории, не только в сообщении агента. Параметры подключения берите из `OPENAI_API_KEY`, `OPENAI_BASE_URL`, `OPENAI_MODEL`; не зашивайте свои адреса, ключи или другую модель. Не используйте `previous_response_id`, серверную историю и `hosted tools`. [Техническая инструкция](#).

Шаг 4 Пройдите демо

1. Откройте [форму участия](#).
2. Введите ссылку на GitHub-репозиторий и полный SHA. Для самого демо имя автора и Telegram не требуются.
3. Нажмите «**Проверить демо · 2 проверки**».
4. Дождитесь завершения обеих проверок. У них общий лимит около 10 минут плюс проверка результата. Пока идёт демо, сохраняйте эту вкладку открытой.
5. Прочитайте результат и подсказки. Если меняете код, сделайте новый коммит, отправьте его на GitHub, замените SHA в форме и повторите проверку.

Первая проверка использует небольшой Python-проект. Вторая проверяет более крупный контекст и на первом запросе специально возвращает тестовый `incomplete` из-за лимита выходных токенов. Агент должен пережить такой ответ и продолжить работу. Это тест устойчивости, а не признак того, что сайт сломан. Всего разрешено **3 запуска демо за 24 часа на IP**, одновременно только один. Не расходуйте попытки на незакоммиченный или недоступный код. [Как устроено демо](#).

Если демо не проходит

Что видно	Что проверить
Репозиторий или коммит недоступен	Public, полный SHA, push нужного коммита, отсутствие опечаток в URL.
Не найден <code>agent.json</code> или код не запускается	Файлы в корне, валидный JSON, пути в <code>entrypoint</code> , чтение <code>--task-file</code> и работа в <code>--workspace</code> .
Нет зависимости	В демо зависимости автоматически не устанавливаются из вашего <code>lock</code> -файла. Сверьте доступное окружение с инструкцией; для заготовки используется <code>openai==2.26.0</code> .
Ошибка JSON после первого ответа	Сначала проверяйте статус завершения. Не передавайте пустой или обрезанный текст в JSON-парсер.
Слишком большой контекст или HTTP 413	Используйте поиск, чтение фрагментов и ограниченный вывод инструментов. Не повторяйте тот же переполненный запрос бесконечно.
Цикл не заканчивается	Лимиты шагов, времени и повторных попыток; условия завершения после ответа без вызовов.
Агент написал «исправлено», но изменений нет	Реальные записи в <code>/testbed</code> , корректный путь, фактический diff. Итоговый текст не заменяет изменения файлов.

Окружение демо ограничено: в частности, Python 3.11.5 и `openai==2.26.0`; произвольный Dockerfile не собирается. Lock-файл нужен для воспроизводимости, но не означает, что демо автоматически установит любые зависимости. Перед нестандартным стеком прочитайте полный [перечень ограничений](#).

Шаг 5 Отправьте заявку

Заполните поля формы:

Поле	Что указать
Название агента	Понятное название вашего проекта.
Автор	Ваше имя.
GitHub-репозиторий	Ссылка на публичный репозиторий агента.
Полный commit SHA	Те же 40 символов, на которых проверяли финальную версию.
Модель и версия	На сайте зафиксирована <code>deepseek-v4.1-flash</code> ; менять её не нужно.
Контакт для согласования	Ваш Telegram, например <code>@username</code> .

Прочитайте согласие на публикацию названия, репозитория и результатов. Контакт для согласования не публикуется. Если согласны, отметьте чекбокс и нажмите **«Отправить на проверку»**. Дождитесь подтверждения отправки и сохраните репозиторий и SHA отправленной версии. [Форма участия](#).

Результат демо связывается с заявкой при совпадении репозитория и SHA. Отправляйте её из той же вкладки, в которой запускали проверку. Статус демо хранится с приватным токеном этой вкладки; не передавайте его другим. [Привязка демо к заявке](#).

Различайте три результата. Демо проверяет работоспособность и устойчивость. Заявка фиксирует вашу версию для рассмотрения. Основной SWE-bench-прогон проводится после допуска организатором. Сайт позволяет отправить заявку даже без успешного демо, но критерий домашки из курса — пройти демо; отправка формы сама по себе не означает выполнение задания или получение оценки. [Порядок участия](#).

Перед отправкой

- [] Публичный репозиторий содержит ваши изменения, корневой `agent.json`, README и lock-файл.
- [] Секретов нет ни в файлах, ни в истории коммитов.
- [] Коммит отправлен на GitHub; в форме указан его полный SHA.
- [] Агент использует инструменты для поиска и чтения фрагментов, не загружает весь репозиторий одним запросом.
- [] Есть ограничение шагов, времени и повторных попыток; обработан `incomplete`.
- [] Обе проверки демо пройдены на том же SHA, который отправляется в заявке.
- [] Telegram указан верно; получено подтверждение отправки формы.

Правила и интерфейс сверены 9 октября 2026. При расхождениях по ограничениям стенда приоритет у [актуальной технической инструкции](#).

Что читать и смотреть после второго занятия

Подборка о structured output, инструментах и skills. Начните с трёх руководств ниже; остальные материалы выбирайте под свою задачу. Время чтения — ориентир для указанных разделов, а не длительность видео.

Все ссылки ведут к авторам платформ или организаторам лекций. Тексты на русском, но код и названия API на английском. Документация Claude и GigaChat объясняет общие механизмы: названия полей и методы SDK нельзя механически переносить в Responses API из практики или в шлюз Planerverse.

Основной маршрут

1 Структурированные выходные данные

[Claude Platform Docs на русском](#). Руководство, ориентир 15–20 минут на основные разделы.

Читайте введение, работу схемы, примеры с Pydantic и ограничения, особенно незавершённый ответ при достижении лимита токенов. Свяжите это с тремя уровнями проверки из практики: JSON, схема, факты. В Claude используется свой API формата ответа; в коде курса — `text.format` и `responses.parse`.

После чтения: объясните, почему схема не доказывает, что указанный моделью файл существует, и почему обрезанный ответ нельзя считать корректным объектом.

2 Как работает использование инструментов

[Claude Platform Docs на русском](#). Руководство, ориентир 10–15 минут.

Разберите цикл: описание инструмента, предложение вызова моделью, исполнение клиентской программой, возврат результата и следующий ответ. Сопоставьте его с `04_tool_call.py`. В Claude названия блоков отличаются от `function_call` и `function_call_output` в практике, но разделение ответственности остаётся тем же.

После чтения: нарисуйте два запроса к модели и отметьте место, где выполняется настоящий Python-код. Что получит модель, если программа отклонила вызов?

3 Skills в Claude Code

[Расширение Claude с помощью skills](#). Документация на русском, ориентир 15–20 минут на создание первого skill, frontmatter и вспомогательные файлы.

Посмотрите устройство `SKILL.md`, роль `description`, размещение файлов и загрузку справочных материалов по необходимости. Это полезно для дополнительного варианта домашки. Функции конкретного продукта и дополнительные поля Claude Code не являются универсальным контрактом всех агентов.

После чтения: создайте skill для своей задачи и проверьте три запроса: должен включиться, не должен включиться, не хватает инструмента или доступа. Отделяйте качество инструкции от разрешений среды.

Если хочется глубже

4 Контракт инструмента через JSON Schema

[Описание пользовательских функций, GigaChat](#). Справочник на русском, ориентир 15 минут на описание функции и примеры параметров.

Полезен для проектирования `name`, `description`, `parameters`, обязательных полей и допустимых значений. Сравните хорошее описание аргумента с расплывчатым «любой текст». Специфические поля GigaChat не нужно добавлять в инструменты курса без проверки API.

Попробуйте описать инструмент `search_code`: какие аргументы обязательны, чем ограничить вывод и как объяснить модели, когда поиск лучше чтения целого файла?

5 Видео об инструментах и MCP

[Agents Week, лекция 1.2 Tools. MCP, Алёна Зайцева](#). Страница организаторов с YouTube и ВКонтакте.

Русская речь, презентация на английском. Подходит, если уже понятен базовый tool loop и хочется связать его с подключением внешних систем. MCP здесь дополнительная тема, он не требуется для сдачи домашки. На странице интенсива есть и вводная лекция 1.1, если хочется повторить основы.

Вопрос после просмотра: чем транспорт подключения инструментов отличается от решения модели вызвать конкретную функцию?

6 Видео о памяти и ограничениях агента

[Agents Week, лекция 2 Memory and Guardrails in LLM-Powered Agents, Кирилл Мищенко](#). На странице программы выберите лекцию 2; рядом доступны записи на YouTube и ВКонтакте, а также отдельный семинар 2.

Русская речь, слайды на английском. Этот блок полезен при переходе от маленького учебного проекта к длинной истории и реальным репозиториям. Смотрите после базового цикла, не вместо его реализации.

Практический вопрос: что оставить в истории после чтения длинного файла и как сократить её, не потеряв задачу, ограничения и связь вызовов с результатами?

7 Прикладной вебинар с function calling

[Создание Telegram-бота на базе LLM с RAG и Function Calling, Дмитрий Сошников](#). Запись вебинара Yandex Cloud от 3 апреля 2025; на странице есть встроенное видео и ссылка на YouTube.

Материал на русском для тех, кто хочет увидеть использование внешних систем в другом приложении, а не только coding-агенте. В нём также есть RAG и многоагентное тестирование: для второго занятия это необязательное расширение. Примеры используют сервисы Yandex, а не API домашки.

Вопрос после просмотра: какие действия безопасно разрешать автоматически, а какие требуют проверки аргументов или подтверждения пользователя?

Как извлечь пользу из одного материала

Выберите одну идею, внесите одно изменение в своего агента и проверьте его на конкретном примере. Запишите ожидаемое поведение, фактические вызовы, результат и ограничение, которое осталось. Описание модели «всё исправлено» не заменяет проверку файлов и тестов.

Для сдачи ориентируйтесь не на примеры чужих платформ, а на [техническую инструкцию Planerverse](#).